IND320-1 26H Data til beslutning

Student name: Lovisa Nguyen

Streamlit App: https://reservoir-app-datatodecision-v2026.streamlit.app/

Github Resipotory: https://github.com/myelovis/reservoir_streamlit_app

Screencast: https://www.dropbox.com/scl/fi/d194lahhvyvra1ap45xm2/Hydropower_Dashboard.mp4?rlkey=z2fd64go8jsa6p8g0zpqyxzk5&st=wyl1b088&dl=0

Task

- *Jupyter Notebook* Read the reservoirs.csv file using Pandas (located in D2Dbook/data). Rename headers to English and understandable. Print its contents in a relevant way. Plot each column separately. Plot all columns together. Consider how to make this natural, given that the scales are different. Remember to fill in the log and AI mentioned in the General section above.

- *Streamlit app* Create a Streamlit app including: requirements.txt or uv.lock (for package dependencies) Four pages (with dummy headers and test content for now) with separate .py files. The front/home page should have a sidebar menu with navigation options to the other pages. On the second page: A table showing the imported data (see below). Use the row-wise LineChartColumn() to display the first month of the data series. There should be one row in the table for each column of the imported data. On the third page: A plot of the imported data (see below), including header, axis titles and other relevant formatting. A drop-down menu (st.selectbox) choosing any single column in the CSV or all columns together. A selection slider (st.select_slider) to select a subset of the months. The default should be the first month. Data should be read from a local CSV-file (reservoirs.csv), using caching for app speed. In part 2 of the project, we will remove this file and rely on MongoDB instead.

- *Screencast* 5-minute screencast where you briefly showcase your running app and explain what your code does. Canvas does not permit upload of more than one file in the assignment. If you make a screencast in Panopto, you can share a link to your video and add the link to the Jupyter Notebook or as a submission comment (make sure the file permissions are open for anyone). If you use a different tool for making the screen cast, you can upload your video to the GitHub repo.

Project Log & Development Summary

This project centered on building and refining a multi-page **Streamlit Web Application** for visualizing and analyzing historical Norwegian hydropower reservoir data. The development process focused on structuring the application cleanly across separate pages, resolving runtime scope errors, creating custom Matplotlib data visualizations, and crafting a polished, dark-themed UI/UX.

Key Milestones & Coding Support

1. **Solving Notebook & Scale Visualization Challenges**

- **Initial Notebook Logic:** Formulated code to analyze time-series metrics from `reservoirs.csv`.
- **Scale Mismatch Solutions:** Developed a dual y-axis Matplotlib plot (TWh capacity/storage vs. 0–1 fill ratios) and a Min-Max normalization pipeline to naturally plot metrics with vastly different numeric scales on a unified canvas.

2. **Resolving Streamlit Multi-Page Scope Errors (`NameError: name 'df' is not defined`)**

- **Diagnosis:** Identified that Streamlit subpages in the `pages/` directory execute as independent, isolated scripts and do not automatically inherit variables from entry scripts.
- **Implementation:** Standardized dataset loading across pages by implementing a central loader in `utils.py` using `@st.cache_data`. Integrated explicit `df = load_data()` instantiation within `try/except` safety blocks in every page script.

3. **Building Advanced Visualization Modules (`3_Summary_Analytics.py`)**

- **Grid Subplot Architecture:** Implemented a publication-quality `plot_hydropower_analytics` plotting function. Added custom area shading for change/delta metrics (green for inflow/fill, red for outflow/drain) to eliminate visual noise.
- **Smart Axis Formatting:** Incorporated `matplotlib.dates` (`AutoDateLocator` and `ConciseDateFormatter`) for clean x-axis date rendering across multi-year time series.
- **Combined Overview Section:** Appended a national-level aggregation block (`df_national`) with dual y-axes to present aggregated capacity, storage, and fill ratio dynamics beneath the feature grid.

4. **UI/UX Refinement & Theme Blending**

- **Custom Canvas Backgrounds:** Configured Matplotlib `rcParams` (`figure.facecolor`, `axes.facecolor`, `savefig.facecolor`, grid lines, and legends) to run on a sleek, high-contrast `#000000` black theme for `2_Interactive_Visualizations.py`.
- **Dynamic Streamlit Theme Matching:** Utilized `st.get_option("theme.backgroundColor")` and `st.get_option("theme.secondaryBackgroundColor")` to automatically extract the active user interface palette, ensuring charts blend into the app container without harsh borders.

5. **Project Documentation**

- Generated a structured `README.md` file outlining features, multi-page project directory architecture, virtual environment setup, dependencies, and execution

instructions.

Import libraries

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
```

Read the reservoirs.csv file using Pandas

In [2]:
```
df = pd.read_csv('~/nmbu/datatilbeslutning/datatilbeslutning/reservoirs.csv')
df.head()
```

Out[2]:

	dato_Id	omrType	omrnr	iso_aar	iso_uke	fyllingsgrad	kapasitet_TWh	fylling_TWh
0	1995-09-03	EL	4	1995	35	0.944721	21.079208	19.913979
1	2020-11-15	EL	1	2020	46	0.974361	6.003264	5.849344
2	2011-08-28	EL	4	2011	34	0.786499	21.079208	16.578772
3	1998-07-05	EL	3	1998	27	0.793969	8.920932	7.082947
4	2011-03-27	EL	4	2011	12	0.300820	21.079208	6.341054

In [3]:
```
df.columns
```

Out[3]:
```
Index(['dato_Id', 'omrType', 'omrnr', 'iso_aar', 'iso_uke', 'fyllingsgrad',
       'kapasitet_TWh', 'fylling_TWh', 'neste_Publiseringsdato',
       'fyllingsgrad_forrige_uke', 'endring_fyllingsgrad'],
      dtype='str')
```

Rename columns

In [4]:
```
df = df.rename(columns={
    'dato_Id': 'date_id',
    'omrType': 'area_type',
    'omrnr': 'area_number',
    'iso_aar': 'iso_year',
    'iso_uke': 'iso_week',
    'fyllingsgrad': 'fill_ratio',           # Reservoir fill level / percentage
    'kapasitet_TWh': 'capacity_twh',       # Total capacity in TWh
    'fylling_TWh': 'storage_twh',          # Stored water/energy in TWh
    'neste_Publiseringsdato': 'next_publication_date',
    'fyllingsgrad_forrige_uke': 'fill_ratio_previous_week',
    'endring_fyllingsgrad': 'fill_ratio_change'
})
```

```
#Verify name changes
df.columns
```

Out[4]:
```
Index(['date_id', 'area_type', 'area_number', 'iso_year', 'iso_week',
       'fill_ratio', 'capacity_twh', 'storage_twh', 'next_publication_date',
       'fill_ratio_previous_week', 'fill_ratio_change'],
      dtype='str')
```

TWh stands for **Terawatt-hour**, which is a unit of **energy**.

In the context of the Norwegian hydropower dataset you are working with:

- **Watt-hour (Wh):** A unit measuring electrical energy.
- **Terawatt-hour (TWh):** $1\text{ TWh} = 1,000,000,000,000\text{ Watt-hours}$ (or 1 billion kilowatt-hours / $1,000\text{ GWh}$).

Why is water measured in energy (TWh)?

In hydropower systems, reservoirs hold water, but energy companies measure that water in terms of **how much electrical energy it can generate** when passed through the turbines downstream.

- `capacity_twh` = Maximum electrical energy potential of the water when the reservoir is 100% full.
- `storage_twh` = Current electrical energy potential stored in the reservoir right now.

In [5]:
```
df.head()
```

Out[5]:

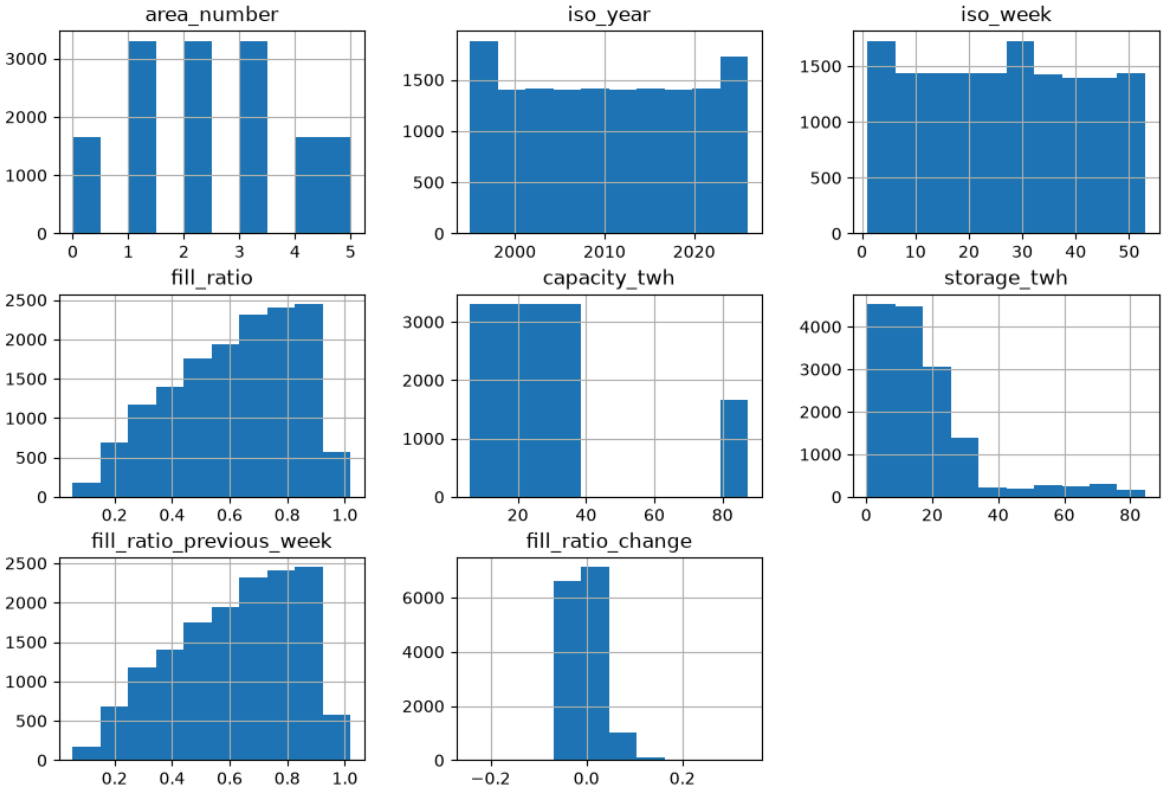
	date_id	area_type	area_number	iso_year	iso_week	fill_ratio	capacity_twh	storage
0	1995-09-03	EL	4	1995	35	0.944721	21.079208	19.91
1	2020-11-15	EL	1	2020	46	0.974361	6.003264	5.84
2	2011-08-28	EL	4	2011	34	0.786499	21.079208	16.57
3	1998-07-05	EL	3	1998	27	0.793969	8.920932	7.08
4	2011-03-27	EL	4	2011	12	0.300820	21.079208	6.34

In [6]:
```
df.describe()
```

Out[6]:

	area_number	iso_year	iso_week	fill_ratio	capacity_twh	storage_
count	14877.000000	14877.000000	14877.000000	14877.000000	14877.000000	14877.000
mean	2.333333	2010.346038	26.405929	0.618852	29.145860	18.139
std	1.490762	9.146696	15.017910	0.214850	22.739070	15.966
min	0.000000	1995.000000	1.000000	0.055160	6.003264	0.331
25%	1.000000	2002.000000	13.000000	0.456234	17.390587	7.321
50%	2.000000	2010.000000	26.000000	0.648049	23.237715	14.501
75%	3.000000	2018.000000	39.000000	0.802135	34.043590	22.203
max	5.000000	2026.000000	53.000000	1.020072	87.437580	84.544

In [7]:
```
# Data distribution
df.hist(figsize=(12, 8))
plt.show()
```



Handling timestamp data

In [8]:
```
# 1. Convert all date/dato columns to datetime
date_cols = df.filter(regex='(?i)date|dato').columns
df[date_cols] = df[date_cols].apply(lambda col: pd.to_datetime(col.astype(str),

# 2. Sort chronologically by the primary date column
if len(date_cols) > 0:
    df = df.sort_values(by=date_cols[0]).reset_index(drop=True)
```

Plot each column separately.

In [9]:
```
# Write a function to plot multiple coloumns
sns.set_theme(style="whitegrid")

def plot_column(df, col_name, date_col='date_id', kind='line', title=None):
    """
    Plots a single column from the DataFrame.

    Parameters:
    - df: pandas DataFrame
    - col_name: str, column to plot on the y-axis
    - date_col: str, column to use for x-axis
    - kind: str, 'line', 'scatter', 'bar', or 'hist'
    - title: str, optional plot title
    """
    plt.figure(figsize=(10, 4.5))

    if kind == 'line':
        x_data = df[date_col] if date_col in df.columns else df.index
        plt.plot(x_data, df[col_name], color='#0066cc', linewidth=1.5)
        plt.xlabel(date_col)
        plt.ylabel(col_name)

    elif kind == 'scatter':
        x_data = df[date_col] if date_col in df.columns else df.index
        plt.scatter(x_data, df[col_name], alpha=0.4, s=12, color='#0066cc')
        plt.xlabel(date_col)
        plt.ylabel(col_name)

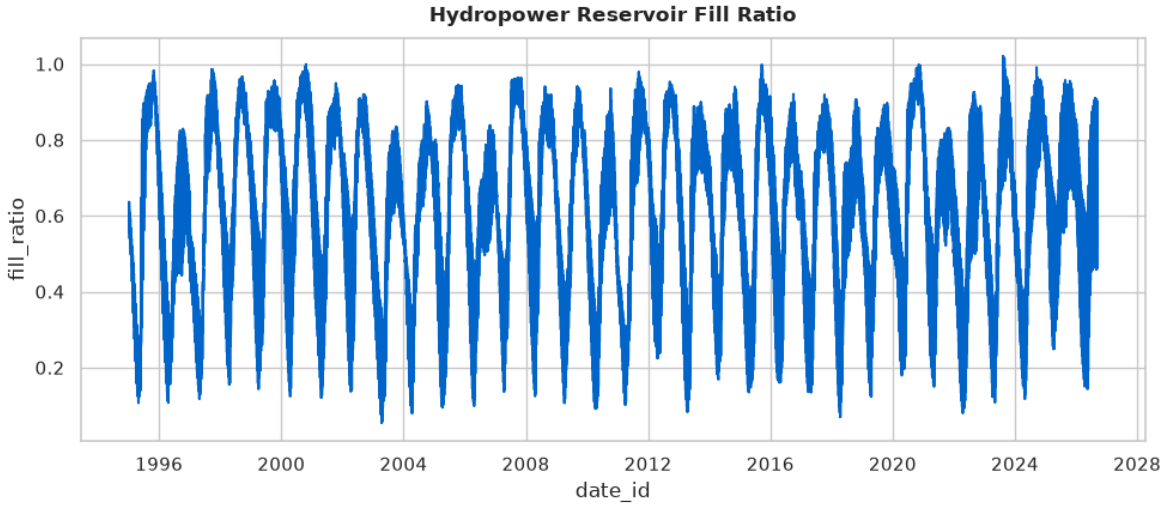
    elif kind == 'bar':
        counts = df[col_name].value_counts()
        sns.barplot(x=counts.index, y=counts.values, palette='viridis')
        plt.xlabel(col_name)
        plt.ylabel('Count')

    elif kind == 'hist':
        sns.histplot(df[col_name].dropna(), kde=True, color='#0066cc')
        plt.xlabel(col_name)
        plt.ylabel('Frequency')

    plot_title = title if title else f"Trend / Distribution of '{col_name}'"
    plt.title(plot_title, fontsize=12, fontweight='bold', pad=10)
    plt.tight_layout()
    plt.show()
```

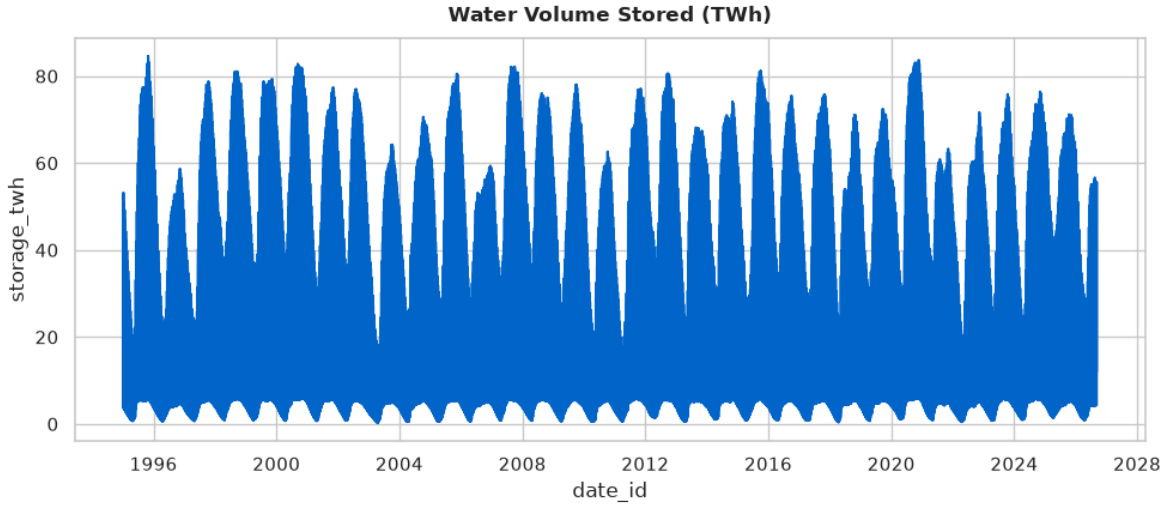
Reservoir Fill Ratio Over Time

In [10]:
```
plot_column(df, col_name='fill_ratio', kind='line', title='Hydropower Reservoir
```



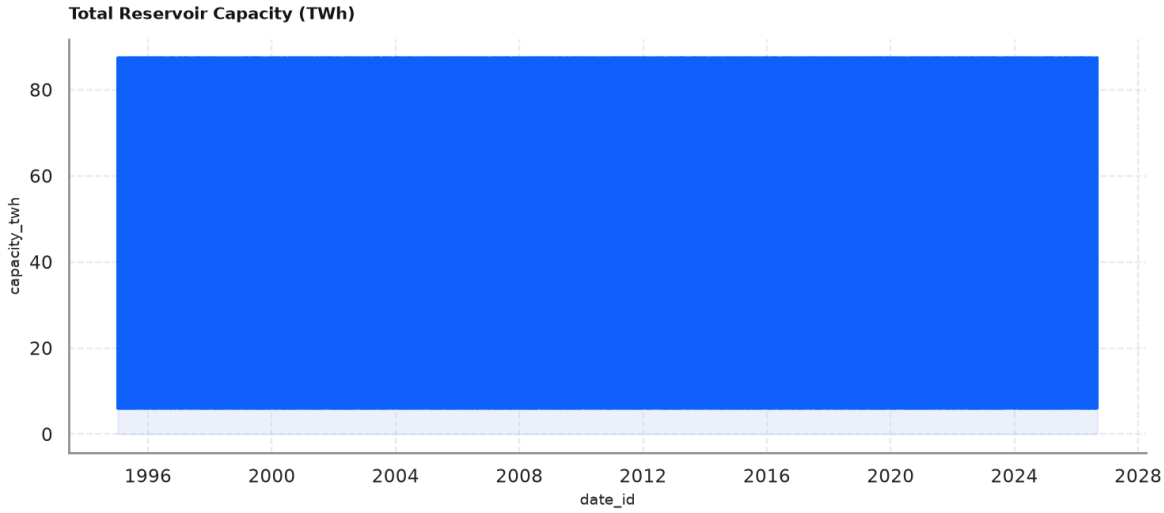
Water Volume Stored (TWh)

In [11]:
```
plot_column(df, col_name='storage_twh', kind='line', title='Water Volume Stored
```



Total Reservoir Capacity (TWh)

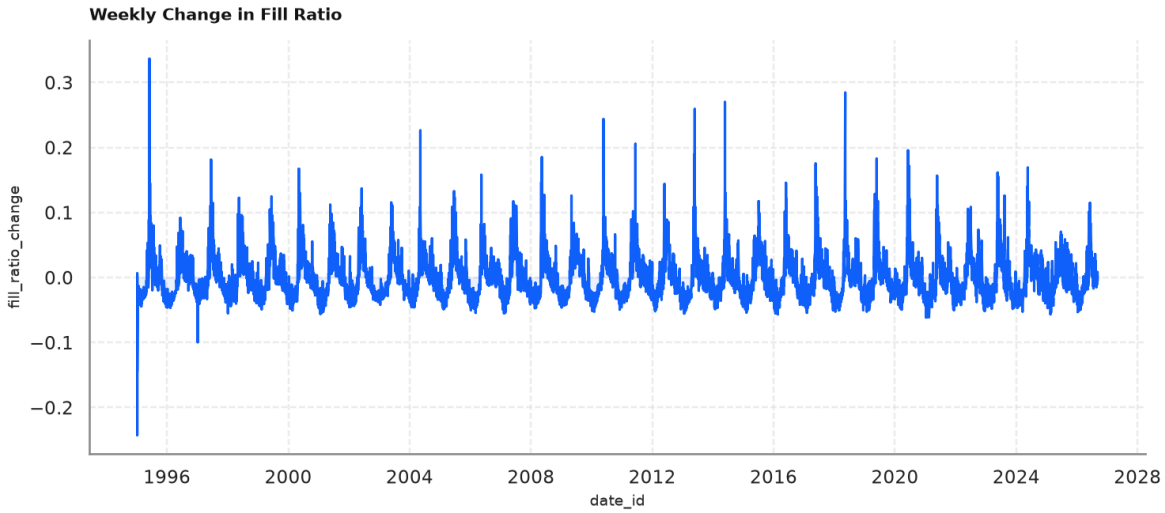
In [29]:
```
plot_column(df, col_name='capacity_twh', kind='line', title='Total Reservoir Cap
```



Weekly Change in Fill Ratio

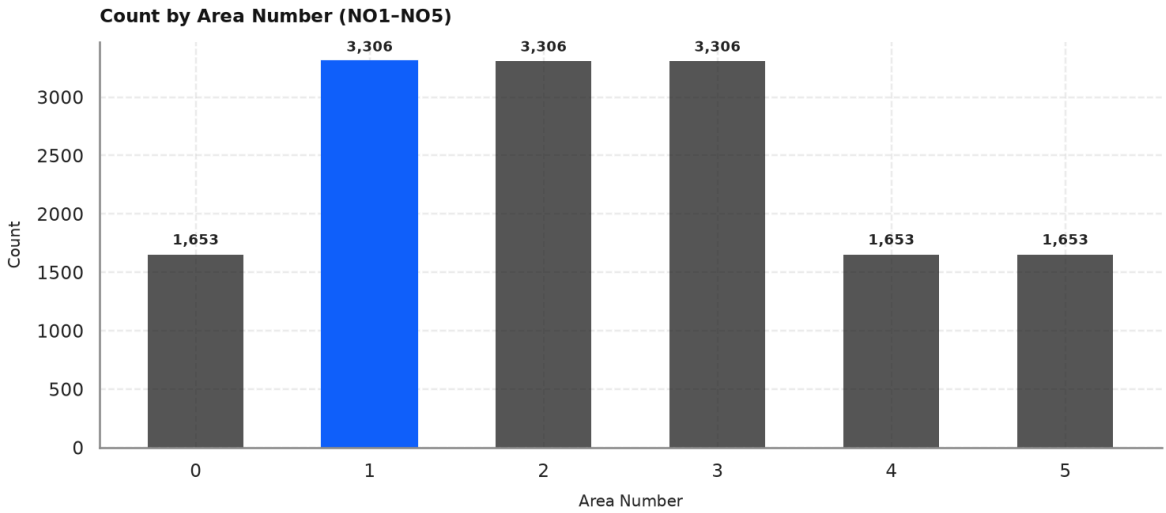
In [28]:
```
plot_column(df, col_name='fill_ratio_change', kind='line', title='Weekly Change
```

```
findfont: Failed to find font weight medium, now using 400.
```



Count by Area Number

In [25]:
```
plot_column(df, col_name='area_number', kind='bar', title='Count by Area Number
```



Plot all columns together

In [26]:
```
# Set global publication-quality defaults
plt.rcParams.update({
    "font.family": "sans-serif",
    "font.size": 9,
    "axes.titlesize": 10,
    "axes.titleweight": "semibold",
    "axes.labelsize": 9,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "grid.linestyle": "--",
    "figure.autolayout": False,
})

def plot_hydropower_analytics(data: pd.DataFrame, date_col: str = "date_id", col
```

```
    df_proc = data.copy()
    if not pd.api.types.is_datetime64_any_dtype(df_proc[date_col]):
        df_proc[date_col] = pd.to_datetime(df_proc[date_col].astype(str), errors

    df_proc = df_proc.sort_values(by=date_col).reset_index(drop=True)
    numeric_cols = df_proc.select_dtypes(include=["number"]).columns.drop(date_c

    num_plots = len(numeric_cols)
    if num_plots == 0:
        raise ValueError("No numeric metrics detected for visualization.")

    n_rows = int(np.ceil(num_plots / cols_per_row))
    fig, axes = plt.subplots(nrows=n_rows, ncols=cols_per_row, figsize=(14, 3.2
    axes_flat = np.atleast_1d(axes).flatten()

    x_axis = df_proc[date_col]

    for idx, col in enumerate(numeric_cols):
        ax = axes_flat[idx]
        title_str = col.replace("_", " ").title()

        # Clean Area Fill for Delta/Change Metrics (Fixes the hairbrush visual e
        if "change" in col or "delta" in col:
            ax.plot(x_axis, df_proc[col], color="#555555", linewidth=0.7)
            ax.fill_between(x_axis, df_proc[col], 0, where=(df_proc[col] >= 0),
            ax.fill_between(x_axis, df_proc[col], 0, where=(df_proc[col] < 0), c
            ax.axhline(0, color="black", linewidth=0.8, linestyle="--")

        # High-Contrast Blue Line + Light Shading for Standard Metrics
        else:
            ax.plot(x_axis, df_proc[col], color="#0F62FE", linewidth=1.4, label=
            ax.fill_between(x_axis, df_proc[col], color="#0F62FE", alpha=0.08)

        # Styling
        ax.set_title(col.replace("_", " ").upper(), loc="left", pad=6)
        ax.set_ylabel(title_str, fontsize=8.5)
        ax.tick_params(axis="both", which="major", labelsize=8)
        ax.spines["top"].set_visible(False)
        ax.spines["right"].set_visible(False)
        ax.legend(loc="upper left", frameon=True, framealpha=0.9, fontsize=8)

    # Smart X-Axis Date Formatting
    locator = mdates.AutoDateLocator(minticks=4, maxticks=8)
    formatter = mdates.ConciseDateFormatter(locator)

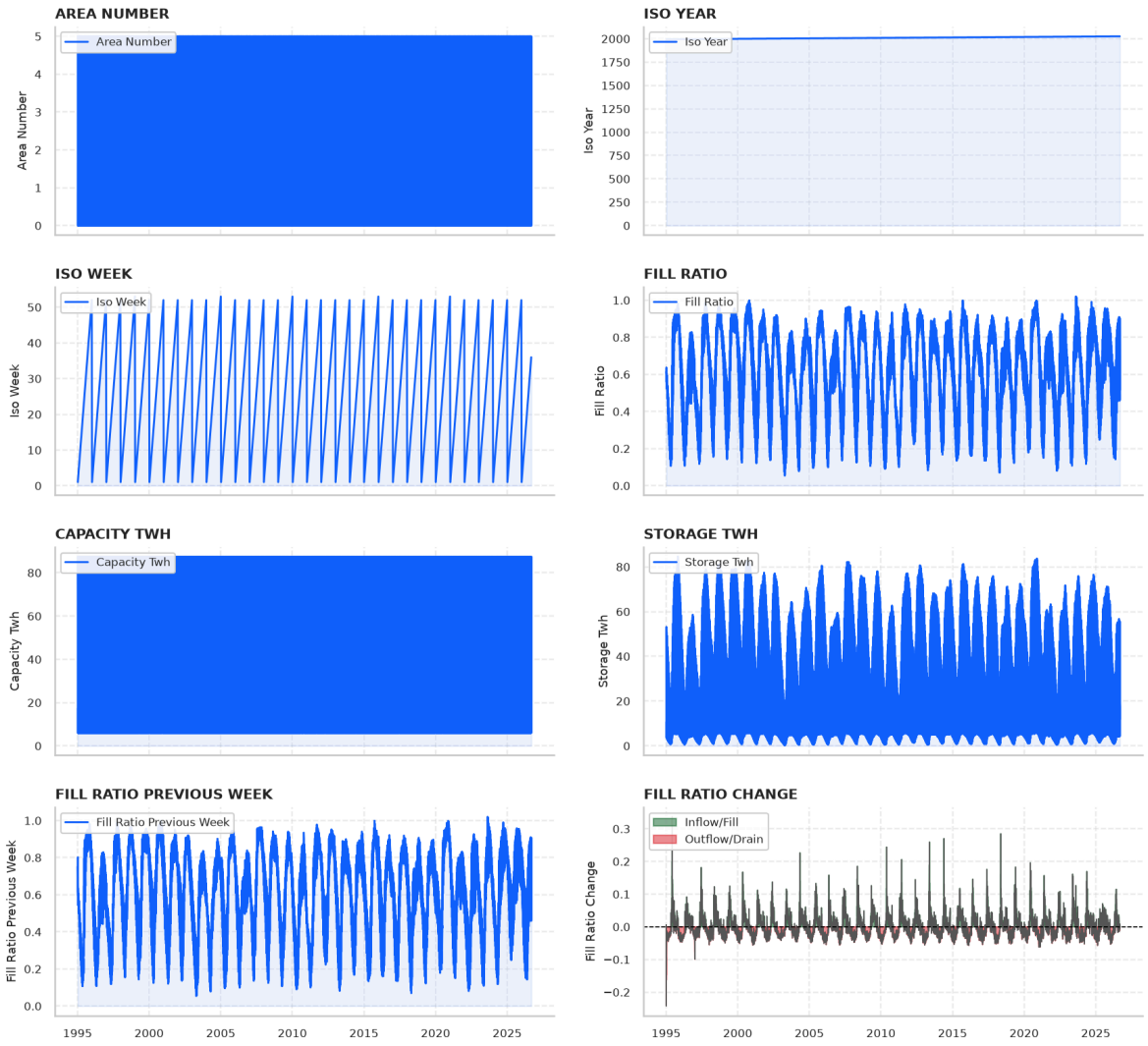
    for ax in axes_flat[:num_plots]:
        ax.xaxis.set_major_locator(locator)
        ax.xaxis.set_major_formatter(formatter)

    # Clean up empty subplots
    for unused_ax in axes_flat[num_plots:]:
        fig.delaxes(unused_ax)

    fig.subplots_adjust(hspace=0.25, wspace=0.18)
    return fig, axes

fig, axes = plot_hydropower_analytics(df, date_col=primary_date)
plt.show()
```

```
findfont: Failed to find font weight medium, now using 400.
```


AREA NUMBER
Area Number
ISO YEAR
Iso Year
ISO WEEK
Iso Week
FILL RATIO
Fill Ratio
CAPACITY TWH
Capacity Twh
STORAGE TWH
Storage Twh
FILL RATIO PREVIOUS WEEK
Fill Ratio Previous Week
FILL RATIO CHANGE
Inflow/Fill
Outflow/Drain
Fill Ratio Change
1995
2000
2005
2010
2015
2020
2025

In [ ]: